NexusPOS – Retail POS System

High-Level Design (HLD) Document

Project	NexusPOS – Retail Point of Sale & Store Management System
Document Type	High-Level Design (HLD)
Version	1.0
Framework	.NET 10.0 (ASP.NET Core Microservices)
Frontend	Angular 21
Status	Released for Review
Prepared By	Architecture Team
Date	15 April 2026

1. Introduction

1.1 Purpose

This High-Level Design (HLD) document describes the overall architecture, component interactions, technology stack, and deployment strategy of the NexusPOS Retail Point of Sale and Store Management System. It is intended for software architects, technical leads, and senior developers who need a bird's-eye view of the system.

1.2 Scope

NexusPOS is a cloud-ready, microservices-based retail management platform built on .NET 10. It supports multi-store operations with centralized administration, real-time notifications, role-based access control, integrated digital payments via Razorpay, and an event-driven checkout and returns workflow.

1.3 Definitions and Acronyms

Term / Acronym	Definition
POS	Point of Sale – the location/system where a retail transaction is completed
HLD	High-Level Design – document describing system architecture
RBAC	Role-Based Access Control
JWT	JSON Web Token – stateless authentication token
OTP	One-Time Password – time-limited numeric code for 2FA
API Gateway	Ocelot-based reverse proxy centralising all upstream routes
MassTransit	Open-source distributed application framework for .NET using RabbitMQ
Saga	Long-running distributed transaction coordinator pattern
SignalR	ASP.NET Core library for real-time WebSocket/SSE communication
EF Core	Entity Framework Core – .NET ORM for SQL Server
Serilog	Structured logging library used across all microservices
Razorpay	Third-party online payment gateway integrated in PaymentService
MRP	Maximum Retail Price of a product
SKU	Stock Keeping Unit – unique product identifier

1.4 Document Overview

The remainder of this document is organized as follows: Section 2 provides the system architecture overview. Section 3 details the component descriptions. Section 4 covers the technology stack. Section 5 describes data flow. Section 6 addresses security. Section 7 covers deployment considerations.

2. System Architecture Overview

2.1 Architectural Style

NexusPOS follows a Microservices Architecture pattern. Each business domain is encapsulated in an independent, deployable service that communicates through a combination of synchronous HTTP calls (via the API Gateway) and asynchronous message passing (via RabbitMQ / MassTransit). This design promotes independent scalability, fault isolation, and team autonomy.

2.2 High-Level Component Diagram

The system is logically divided into the following layers:

- Presentation Layer: Angular 21 Single Page Application (pos-ui)
- API Gateway Layer: Ocelot-based gateway (Port 5000) that routes, authenticates and forwards all client requests
- Business Services Layer: Eight independent microservices, each with its own database
- Messaging Layer: RabbitMQ message broker enabling asynchronous event-driven communication
- Data Layer: Separate SQL Server databases per service (Database-per-Service pattern)
- Shared Infrastructure: RetailPOS.Common library, RetailPOS.Contracts library

2.3 Microservices Overview

Service	Port	Responsibility	Database
IdentityService	5001	Authentication, RBAC, JWT token issuance, OTP, Google OAuth, Refresh Tokens	IdentityDb
CatalogService	5002	Product & Category management, stock quantity tracking	CatalogDb
OrdersService	5003	Bill (cart) management, checkout saga orchestration, customer management	OrdersDb
AdminService	5004	Store management, staff onboarding, inventory adjustments, dashboard analytics, reports	AdminDb
PaymentService	5005	Payment processing (Cash/Card/UPI), Razorpay integration, saga payment commands	PaymentDb
ReturnsService	5006	Return initiation, manager approval workflow, return saga	ReturnsDb

Service	Port	Responsibility	Database
NotificationService	5176	Real-time SignalR hub, event-driven notification persistence	NotificationDb
ApiGateway	5000	Ocelot reverse proxy, JWT authentication, CORS, request routing	–

2.4 Shared Libraries

Library	Purpose
RetailPOS.Common	Shared authorization (Permission constants, PermissionHandler, ServiceExtensions), Serilog configuration helper
RetailPOS.Contracts	Shared MassTransit message contracts: Events (OrderEvents, PaymentEvents, StoreEvents, InventoryEvents, UserEvents, CategoryEvents) and Commands (PaymentCommands, CatalogCommands, UserCommands)

3. Component-Level Descriptions

3.1 API Gateway (Ocelot)

The API Gateway is implemented using the Ocelot library and runs on port 5000. It serves as the single entry point for the Angular frontend, routing all requests to downstream services based on path templates defined in ocelot.json. Key responsibilities include:

- Path translation: /gateway/<domain>/<resource> -> downstream service /api/<resource>
- JWT Bearer authentication on protected routes
- CORS policy enforcement (centralized, removing the need for per-service CORS)
- Priority-based route matching for overlapping path templates

3.2 Identity Service

The Identity Service is the authentication and authorization authority for the entire platform. It implements:

- Two-Factor Authentication (2FA): Email/password login followed by a 6-digit OTP sent via SMTP
- Email verification on registration using OTP
- Password reset via OTP
- Google OAuth 2.0 login using Google ID Token validation
- JWT access token generation (1-hour expiry) with embedded claims: userId, email, role, StoreId, StoreCode, ShiftDate, and granular permission codes
- Refresh Token rotation with 7-day expiry, stored in HttpOnly Secure cookie
- Multi-tenant RBAC: Users can have different roles across different stores (UserStoreRole mapping)
- Auto-migration of legacy plain-text passwords to BCrypt on first login
- Publishes UserRegisteredEvent to RabbitMQ on new registration, triggering AdminService onboarding saga

3.3 Catalog Service

The Catalog Service manages the product catalogue and stock levels for all stores. Key features:

- Product CRUD with SKU, barcode, MRP, selling price, tax code, reorder level, and category
- Category management with store-level scoping
- Stock quantity updates via consumed events (SagaDeductStockConsumer, StockAdjustedConsumer, OrderReturnedConsumer)
- Tenant isolation: StoreId extracted from JWT claims via ITenantProvider
- Seed data initialization on startup

3.4 Orders Service

The Orders Service handles the complete billing lifecycle. Key features:

- Cart management: Bills can be in Draft, Held, or Finalized states
- Bill creation with multiple BillItems, customer association, and cashier tracking
- Checkout Saga: Orchestrates the multi-step checkout process via MassTransit state machine (CheckoutStateMachine)
- Finalization: On FinalizeOrderCommand receipt, bill status transitions to Finalized
- Customer management: Lookup and create customers by mobile number
- Consumes payment and return events to update bill state

3.5 Admin Service

The Admin Service provides centralized management and analytics capabilities:

- Store management: Create, update, soft-delete, and restore stores with event publication to IdentityService and CatalogService
- Staff management: View registered staff, manage pending assignments, assign staff to stores with roles
- Onboarding Saga: Listens for UserRegisteredEvent, creates staff profile in AdminDB, and publishes CreateStaffProfileCommand
- Dashboard analytics: Computes today's sales, bill counts, sales change percentage, hourly trends, low-stock alerts, store-wise breakdown
- Sales and Tax reports: Proxies to OrdersService API with JWT forwarding
- Inventory management: Records manual stock adjustments and publishes StockAdjustedEvent

3.6 Payment Service

The Payment Service handles all payment transactions:

- Cash, Card, and UPI payment recording
- Razorpay integration: Creates Razorpay orders and verifies payment signatures
- Saga command consumer: Processes ProcessPaymentCommand from CheckoutSaga and publishes PaymentProcessedEvent or PaymentFailedEvent
- Refund processing: Processes RefundPaymentCommand from CheckoutSaga and ReturnSaga, publishes PaymentRefundedEvent

3.7 Returns Service

The Returns Service manages product return workflows:

- Return initiation by cashier for a specific bill and product
- Return Saga: Coordinates manager approval, payment refund (via PaymentService), and stock restock (via CatalogService) in correct sequence
- Manager approve/reject actions with notes
- Publishes OrderReturnedEvent on completion for dashboard sync

3.8 Notification Service

The Notification Service provides real-time store-level notifications:

- SignalR hub (/notificationHub): Clients join store-specific groups (Store_{storeId}) using JWT claims
- Event consumers: Listens for UserRegistered, StockAdjusted, OrderPlaced, ReturnInitiated, OrderReturned events
- Persists notifications to SQL Server via NotificationRepository
- Broadcasts real-time notifications to connected clients in the relevant store group

4. Technology Stack

Category	Technology	Version / Details
Backend Runtime	.NET / ASP.NET Core	10.0 (net10.0 target framework)
Programming Language	C#	Latest (C# 13)
Frontend Framework	Angular	21.x (SPA, TypeScript 5.9)
API Gateway	Ocelot	Route-based reverse proxy for .NET
ORM	Entity Framework Core	SQL Server provider, Code-First Migrations
Database	Microsoft SQL Server	Per-service dedicated database
Message Broker	RabbitMQ	AMQP, localhost vhost, guest/guest (dev)
Messaging Library	MassTransit	Saga state machines, consumer registration
Real-Time	ASP.NET Core SignalR	WebSockets, store-grouped notifications
Authentication	JWT Bearer + BCrypt	HMAC-SHA256, 1hr access + 7d refresh token
OAuth 2.0	Google.Apis.Auth	Google ID Token validation
Payment Gateway	Razorpay	Order creation + signature verification
Logging	Serilog	Console + rolling file per service, structured
Email	SMTP (EmailService)	Configurable SMTP for OTP delivery
Authorization	Custom RBAC	Permission-code based policies via RetailPOS.Common
API Documentation	Swagger / OpenAPI	SwaggerGen with JWT bearer support per service
Node / UI Build	npm 11 + Angular CLI 21	Angular build toolchain
SignalR Client	@microsoft/signalr	v10.0 – used in Angular frontend

5. Data Flow and Component Interactions

5.1 User Authentication Flow

The authentication flow is a two-step 2FA process:

1. User submits email and password via Angular frontend to POST /gateway/auth/login
2. API Gateway forwards to IdentityService (port 5001). IdentityService validates credentials, auto-upgrades legacy passwords to BCrypt, generates a 6-digit OTP, stores it with a 10-minute expiry, and sends it via EmailService (SMTP).
3. User submits OTP to POST /gateway/auth/verify-login-otp. IdentityService validates OTP, generates JWT access token (1hr) and refresh token (7d). Refresh token stored in HttpOnly cookie.
4. Frontend stores JWT in memory and uses it as Bearer token for all subsequent API calls.

5.2 Checkout (Billing) Flow

The checkout flow is orchestrated by the Checkout Saga (CheckoutStateMachine in OrdersService):

5. Cashier adds products to cart via POST /gateway/orders/bills/cart/items. OrdersService creates/updates a Draft Bill.
6. Cashier finalizes and publishes CheckoutInitiatedEvent with order details and items.
7. Saga transitions to PaymentProcessing state and publishes ProcessPaymentCommand to PaymentService.
8. PaymentService records payment and publishes PaymentProcessedEvent.
9. Saga transitions to StockDeductionProcessing and publishes DeductStockCommand to CatalogService.
10. CatalogService deducts stock and publishes StockDeductedEvent.
11. Saga finalizes: publishes FinalizeOrderCommand (OrdersService marks bill as Finalized) and OrderPlacedEvent (AdminService and NotificationService record/broadcast the event).
12. Compensation flow: If stock deduction fails, Saga publishes RefundPaymentCommand; on PaymentRefundedEvent, Saga transitions to Failed state.

5.3 Returns Flow

The returns flow is orchestrated by the Return Saga (ReturnStateMachine in ReturnsService):

13. Cashier initiates return via POST /gateway/returns, specifying original bill, product, quantity, and reason.
14. ReturnsService creates a Return record and publishes ReturnInitiatedEvent. Saga transitions to AwaitingApproval.
15. OrdersService and NotificationService receive ReturnInitiatedEvent.
16. Manager approves or rejects via POST /gateway/returns/{id}/approve or /reject.
17. On approval: Saga publishes RefundPaymentCommand to PaymentService. PaymentService processes refund and publishes PaymentRefundedEvent.

18. Saga publishes RestockItemCommand to CatalogService. On StockRestockedEvent, Saga publishes FinalizeReturnCommand and OrderReturnedEvent.

5.4 Staff Onboarding Flow

Staff onboarding is managed by the Onboarding Saga (OnboardingStateMachine in AdminService):

19. Staff registers via POST /gateway/auth/register. IdentityService publishes UserRegisteredEvent.
20. AdminService UserRegisteredConsumer and Onboarding Saga receive event. Saga creates a pending staff profile in AdminDb.
21. Admin assigns staff via POST /gateway/admin/staff/assign. AdminService publishes StaffAssignedEvent.
22. IdentityService StaffAssignedConsumer grants the user a UserStoreRole entry with the assigned store and role.

5.5 Asynchronous Event Map

Publisher	Event / Command	Subscriber(s)
IdentityService	UserRegisteredEvent	AdminService (Onboarding Saga), NotificationService
AdminService	StaffAssignedEvent	IdentityService (StaffAssignedConsumer)
AdminService	StoreCreatedEvent	IdentityService, CatalogService
AdminService	StoreUpdatedEvent / StoreDeletedEvent	IdentityService, CatalogService
AdminService	StockAdjustedEvent	CatalogService, NotificationService
OrdersService (Saga)	CheckoutInitiatedEvent	CheckoutStateMachine (self)
OrdersService (Saga)	ProcessPaymentCommand	PaymentService
OrdersService (Saga)	DeductStockCommand	CatalogService
OrdersService (Saga)	FinalizeOrderCommand	OrdersService
OrdersService (Saga)	OrderPlacedEvent	AdminService (dashboard sync), NotificationService
PaymentService	PaymentProcessedEvent	OrdersService (Saga)
PaymentService	PaymentRefundedEvent	OrdersService (Saga), ReturnsService (Saga)
ReturnsService	ReturnInitiatedEvent	OrdersService, NotificationService, ReturnStateMachine (self)
ReturnsService (Saga)	RefundPaymentCommand	PaymentService
ReturnsService (Saga)	RestockItemCommand	CatalogService

Publisher	Event / Command	Subscriber(s)
ReturnsService (Saga)	OrderReturnedEvent	OrdersService, CatalogService, AdminService, NotificationService
AdminService	CategoryCreatedEvent	CatalogService

6. Security Architecture

6.1 Authentication

- Primary: JWT Bearer tokens (HMAC-SHA256, Issuer: RetailPOS) with 1-hour access token lifetime
- Second Factor: OTP (6-digit, 10-minute expiry) sent via email on every login
- OAuth: Google ID Token validation for social login
- Refresh Token: Cryptographically random 64-byte token, stored server-side and in HttpOnly/Secure/SameSite=Strict cookie (7-day expiry, rotated on use)

6.2 Authorization

- Granular permission-code based RBAC defined in RetailPOS.Common.Authorization.Permissions
- Permissions embedded in JWT as 'permission' claims (e.g., orders:create, catalog:manage)
- PermissionHandler evaluates ASP.NET Core policies against JWT permission claims
- Multi-tenant: StoreId claim scopes data to the user's assigned store. Admin users have null StoreId for cross-store access

6.3 Permission Hierarchy

Role	Key Permissions
Admin	all:all, users:manage, admin:stores:manage, admin:reports:view, admin:inventory:adjust, catalog:manage, catalog:delete
Manager	orders:view_all, orders:void, returns:approve, admin:reports:view, catalog:manage
Cashier	orders:create, orders:view, orders:finalize, orders:hold, payments:create_order, returns:initiate, catalog:view

6.4 Communication Security

- All inter-service communication within the same host uses HTTP (development). Production deployments should use HTTPS with TLS termination at the gateway or service mesh.
- RabbitMQ credentials are currently set to guest/guest for development; should be rotated in production.
- JWT signing key is a configurable symmetric secret; must be rotated per environment.

7. Deployment Considerations

7.1 Current Development Configuration

Service	Port	Notes
API Gateway (Ocelot)	5000	BaseUrl configured in ocelot.json GlobalConfiguration
IdentityService	5001	CORS enabled (allows any origin) for development
CatalogService	5002	CORS centralized to Gateway
OrdersService	5003	CORS centralized to Gateway
AdminService	5004	CORS centralized to Gateway
PaymentService	5005	Razorpay test keys configurable via appsettings
ReturnsService	5006	CORS centralized to Gateway
NotificationService	5176	Dedicated CORS policy for SignalR (AllowCredentials)
RabbitMQ	5672 / 15672	Guest credentials; management UI on 15672
SQL Server	1433	Per-service databases; each service runs EF migrations on startup
Angular Dev Server	4200	ng serve; proxies /gateway/* to port 5000

7.2 Production Deployment Recommendations

- Containerization: Each microservice should be packaged as a Docker container. A docker-compose.yml or Kubernetes Helm chart should define the service mesh.
- Database: Use managed Azure SQL or SQL Server Always On for production databases. Each service retains its own database to preserve service isolation.
- RabbitMQ: Use RabbitMQ with clustering or Azure Service Bus as the production message broker with proper credentials, TLS, and queues with dead-letter routing.
- API Gateway: Add rate limiting, response caching, and load balancing at the Ocelot gateway or offload to an NGINX/Azure API Management layer.
- Identity: Rotate JWT signing key per environment. Store secrets in Azure Key Vault or equivalent.
- Logging: Centralize Serilog output to Elasticsearch/Kibana or Azure Monitor instead of local log files.
- CORS: Lock down to specific production domains instead of AllowAnyOrigin.
- SignalR Scale-out: Configure a Redis backplane for SignalR when deploying multiple NotificationService instances.
- CI/CD: Automate migrations via dotnet ef database update in CI pipeline or migration runner sidecar container.

7.3 Scalability Patterns

- Stateless services: JWT-based auth means any service instance can serve any request without session affinity (except SignalR, which requires sticky sessions or Redis backplane).
- Database-per-service: Allows each service to be scaled, migrated, and deployed independently.
- Saga persistence: Checkout and Return sagas persist state in SQL Server, enabling saga resumption after restarts.
- Consumer scaling: MassTransit consumers can be scaled horizontally; RabbitMQ distributes messages across consumer instances.

8. Cross-Cutting Concerns

8.1 Logging

All microservices use Serilog configured through the shared RetailPOS.Common.Logging.SerilogConfiguration extension. Each service writes structured logs to the console and to daily rolling log files in a Logs/ directory. Log output is enriched with service name, timestamp, and exception details. The minimum log level is Verbose with Microsoft framework logs suppressed to Warning.

8.2 Error Handling

All services use a custom ExceptionMiddleware to intercept unhandled exceptions and return standardized JSON error responses. The middleware maps domain-specific exceptions (NotFoundException, BusinessRuleException, ValidationException, ConflictException) to appropriate HTTP status codes (404, 422, 400, 409).

8.3 Multi-Tenancy

Store-level data isolation is implemented through an ITenantProvider interface. The TenantProvider extracts the StoreId claim from the HTTP context JWT and injects it as a query filter on EF Core DbContext operations. Global Admin users (null StoreId) can bypass filters using IgnoreQueryFilters().

8.4 Repository Pattern

All services implement a generic repository pattern (IGenericRepository<T>) providing standard CRUD operations. Service-specific repositories extend this with domain-specific queries. The pattern abstracts EF Core from the business logic layer.

9. Assumptions and Open Items

ID	Item	Details
A-01	SMTP Configuration	The EmailService requires valid SMTP credentials in appsettings.json. Test accounts bypass email and log OTPs to console.
A-02	Razorpay Test Mode	PaymentService uses Razorpay test keys. Production deployment requires live keys and webhook configuration.
A-03	RabbitMQ Local	All services assume RabbitMQ is running on localhost with guest/guest. Production requires clustered setup with credentials in secrets manager.
A-04	SQL Server Migrations	Each service auto-applies EF Core migrations on startup. If migration fails, the service logs a warning but continues (Identity, Catalog).
A-05	Single-Host Dev	The current ocelot.json hardcodes downstream hosts as localhost. Production requires DNS-based service discovery.
A-06	RefundAmount Calculation	ReturnSaga uses a placeholder calculation for refund amount (quantity * 10). Final implementation should look up actual payment amounts.
A-07	Report Data	ReportService in AdminService proxies directly to OrdersService via HTTP, propagating JWT. This creates a synchronous coupling that should be replaced with event-sourced views in production.
A-08	Google OAuth	GoogleLoginAsync requires Anthropic Google:ClientId in IdentityService appsettings.